



AALBORG
UNIVERSITY

Database Systems

Lecture 11: Recovery

Yumeng Song

Department of Computer Science

Aalborg University

yumengs@cs.aau.dk

Fall 2025

Center for Data-Intensive Systems

Motivation



- System failures happen: power loss, crash, or software bug
 - Data in main memory is lost after crash
 - Transactions may be half-done, leaving DB inconsistent
 - Users expect data to stay correct after restart
- Fixing data manually is slow and costly
- Recovery keeps consistency and availability even when failures occur

Learning Objectives



- After completing this lecture, you will be able to:
 - Connect recovery concepts with what you learned in transaction management
 - Understand how logging records all actions for possible redo or undo
 - Learn basic logging algorithms used in recovery
 - Understand why atomicity matters when a transaction fails halfway
 - See how durability guarantees that committed results survive crashes

Agenda



- Transaction Review
- Failure Classification
 - Transaction failure
 - System crash
 - Disk failure
- Log-Based Recovery
 - Log Entries
 - Recovery Process

Transaction Review



- An example bank transfer
 - I. Read the account balance of A into variable a: *READ(A, a);*
 - II. Reduce account balance by 500 kr.: $a \leftarrow a - 500;$
 - III. Write the new account balance into the database: *WRITE(A, a);*
 - IV. Read account balance of B into variable b: *READ(B; b);*
 - V. Increase account balance by 500 kr.: $b \leftarrow b + 500;$
 - VI. Write new balance into the database: *WRITE(B; b);*

- All steps must be treated as a unit: “All or nothing”.
- Once completed, the changes should be stored permanently.



- A **transaction** is the execution of a sequence of one or more operations (e.g., SQL queries) on a database to perform some higher-level function.

Partial transactions are not allowed!

Transaction Review



- Characteristics of transactions: ACID properties
- Atomicity
 - Either all operations of the transaction are properly reflected in the database or none are.
 - Example: If a debit is made successfully from one account, the corresponding credit is made to the other account.
- Consistency
 - Execution of a transaction in isolation preserves the consistency of the database.
 - According to constraints, checks, assertions
 - Example: Fund transfers should not generate or destroy money -- the overall sum is the same before and afterwards

Transaction Review



- Characteristics of transactions: ACID properties
- Isolation
 - Each transaction appears to have the DB exclusively on its own.
 - Intermediate results must be hidden for other transactions.
 - Often implemented via locks
 - Example: Another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.
- Durability
 - Updates of successfully completed transactions must not get lost despite system failures.
 - Example: The changes made to each account will not be reversed.

Agenda

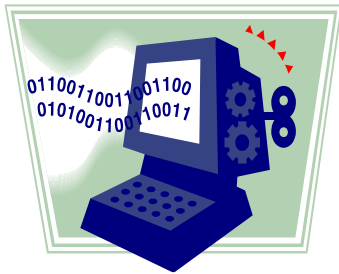


- Transaction Review
- Failure Classification
 - Transaction failure
 - System crash
 - Disk failure
- Log-Based Recovery
 - Log Entries
 - Recovery Process

Failure Classification



- Computer systems are prone to failures
 - May lead to data loss
- Common causes:
 - Software errors
 - Disk failures
 - Power outages
 - Server room issues
 - Human mistakes



Failure Classification



- Transaction failure
- System crash
- Disk failure

Failure Classification



- Transaction failure
 - A transaction fails before reaching its expected end
 - Logical error
 - ◆ Caused by **internal conditions** within the transaction execution
 - ◆ Example:
 - Invalid input: User enters a negative salary value when only positive numbers are allowed
 - Missing data: The program tries to access a record that does not exist
 - Overflow: A division by zero occurs in a computation
 - Integrity constraint violation: A constraint (e.g. unique ID) is violated
 - System error
 - ◆ Caused by **external conditions** during transaction execution
 - ◆ Example:
 - A transaction is rolled back because of a deadlock
 - The DBMS forces termination due to memory shortage or timeout

Failure Classification



- System crash
 - Any event that forces the system to stop running
 - The database itself is **not damaged**, only the in-memory data becomes **inconsistent**
 - Affects **all active transactions**
 - Example:
 - ◆ CPU malfunction

A hardware bug causes the processor to freeze during query execution
 - ◆ Operating system crash

Windows or Linux kernel panic stops the DBMS process
 - ◆ DBMS failure

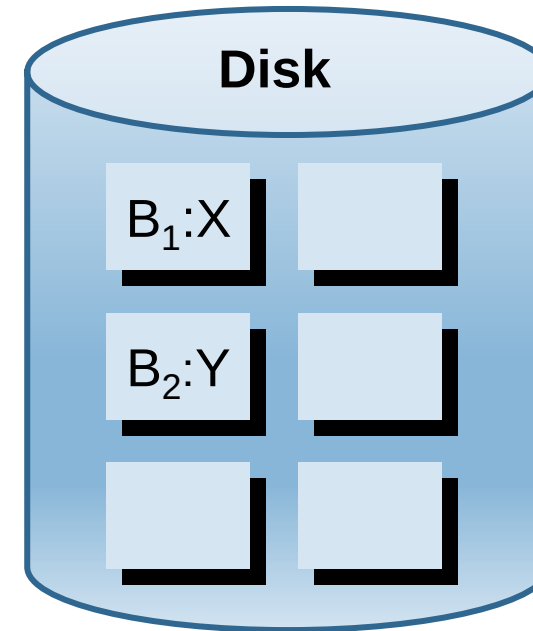
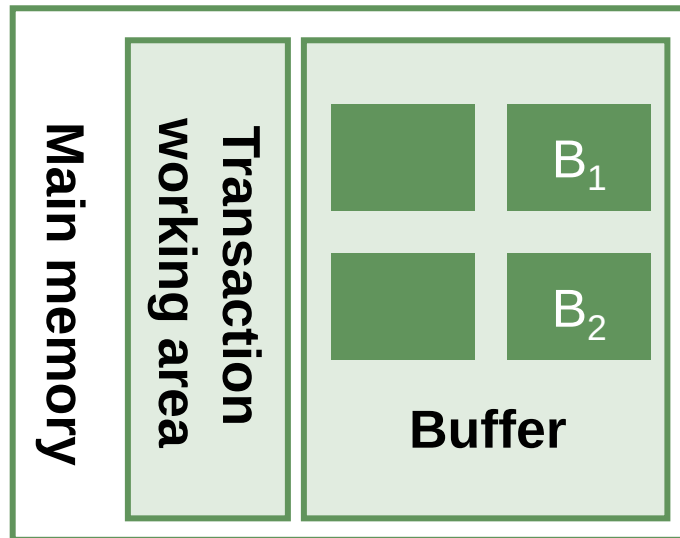
The database server process terminates unexpectedly due to a segmentation fault
 - ◆ Power outage

The server suddenly loses power in the middle of transaction commits

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
 - Data is stored and managed in fixed-size pages (or blocks).
 - ◆ Physical blocks are those blocks residing on the disk.
 - ◆ Buffer blocks are the blocks residing temporarily in main memory.

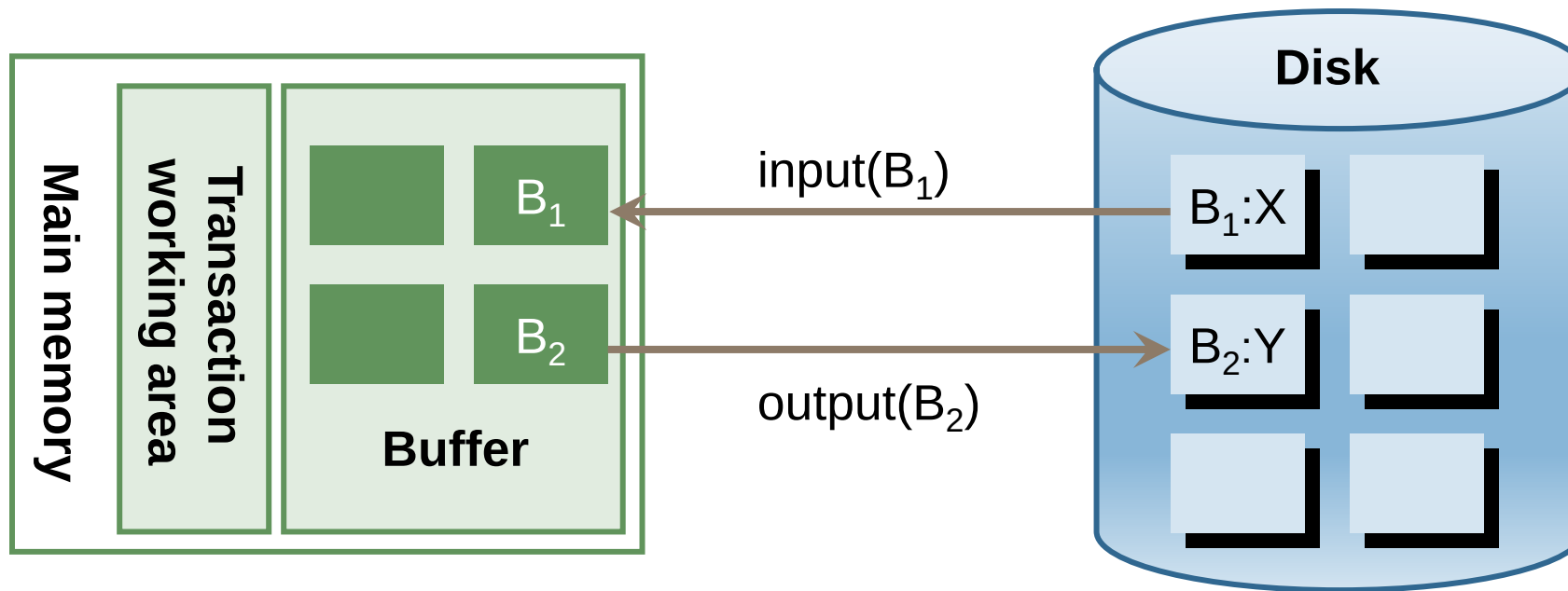


- ◆ Volatile storage (main memory buffer)
- ◆ Non-volatile storage (hard disk)
- ◆ Stable storage (RAIDS, remote backups, . . .)

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
 - Block movements between disk and main memory are initiated through:
 - ◆ input (B) transfers the physical block B to main memory.
 - ◆ output (B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.

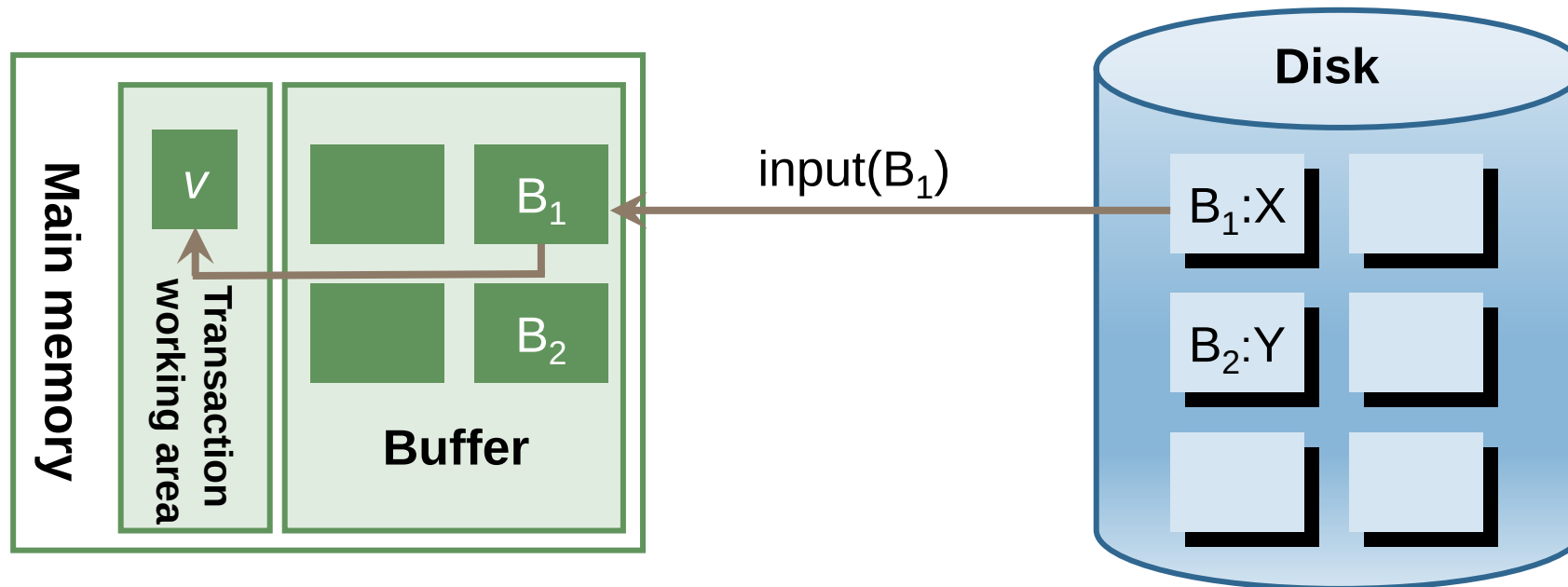


For simplicity, we assume that each data item fits in, and is stored inside, a single block.

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
 - input (B) transfers the physical block B to main memory.
 - ◆ Executed during the operation $READ(X, v)$ (Assigns the value of data item X to variable v)
 - If X is not already in main memory, load block B_1 from disk (input(B_1)))
 - Assign v the value of X stored in buffer block B_1

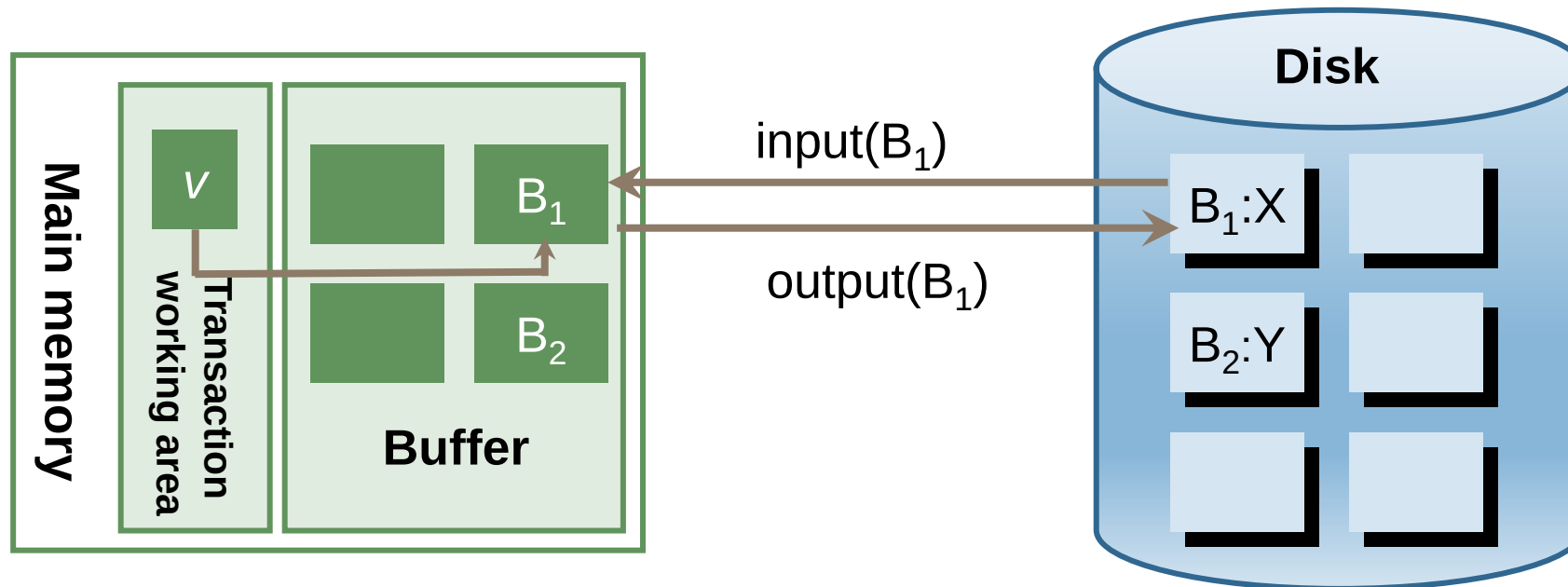


For simplicity, we assume that each data item fits in, and is stored inside, a single block.

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
 - input (B) transfers the physical block B to main memory.
 - ◆ Executed during the operation $WRITE(X, v)$ (Assigns the value of variable v to data item X)
 - If X is not already in main memory, load block B_1 from disk (input(B_1))
 - Assign v the value of X stored in buffer block B_1
 - The buffer manager asynchronously outputs the value to disk **later***

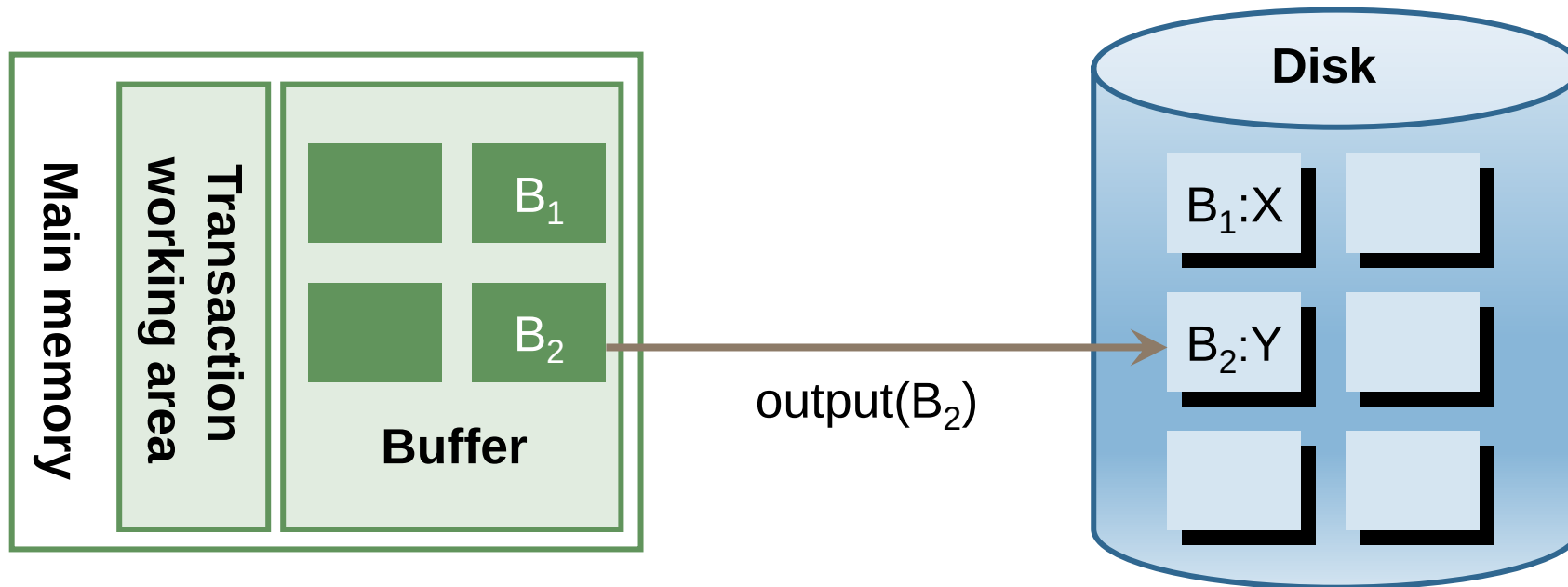


For simplicity, we assume that each data item fits in, and is stored inside, a single block.

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
 - output (B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.
 - ◆ Two situations where output(B) may occur:
 - When the buffer manager **needs free space**
 - When the DBMS **forces the block to be written** to disk



For simplicity, we assume that each data item fits in, and is stored inside, a single block.

Failure Classification



- Why does a system crash cause inconsistency? -- Two-level storage hierarchy
- Example:
 - Normal case:
 - ◆ A transaction changes data in main memory
 - ◆ Data is written to the hard disk
 - ◆ Transaction commits
 - ✓ User assumes that the transaction was successfully completed and all its changes are persistently stored in the database.
 - When a failure occurs:
 - ◆ A transaction changes data in main memory
 - ◆ While writing data to disk, a crash or power outage occurs either before or during the execution of output(B)
 - ◆ None or only part of the data is written to disk
 - ◆ Transaction commits
 - ✗ The user still believes that the transaction was successfully completed and all its changes are persistently stored in the database.

Failure Classification



- Disk Failure
 - External failure:
 - ◆ The database (or part of it) becomes physically damaged
 - Impact:
 - ◆ All transactions **accessing that part of the data** are affected
 - Example:
 - ◆ A disk head crash destroys several blocks of a table storing customer records
 - ◆ Some transactions reading or updating those records fail completely
 - ◆ Recovery must rely on a backup copy or mirrored disk

Recovery



- "Problems" with transactions
 - Atomicity: Transactions may abort (rollback)
 - Durability: What if a DBMS crashes?
- The DBMS ensures that a transaction
 - Either completes and has a permanent result (committed transaction)
 - Or has no effect at all on the database (aborted transaction).
- The role of the recovery component is to ensure **atomicity** and **durability** of transactions in the presence of system failures.

Recovery



- Transaction failure (failure of a not yet committed transaction)
 - Undo the changes of the transaction
- System crash (failure with main memory loss)
 - Changes of committed transactions must be preserved
 - Changes of all non-committed transactions need to be undone
- Disk failure (failure with hard disk loss)
 - Recovery based on archives or backups.
 - The damaged disk is replaced, and data is restored from the most recent backup.
 - Recent updates that occurred after the backup are then reapplied to restore the database to its latest consistent state.

Log-based Recovery

Agenda



- Transaction Review
- Failure Classification
 - Transaction failure
 - System crash
 - Disk failure
- Log-Based Recovery
 - Log Entries
 - Recovery Process

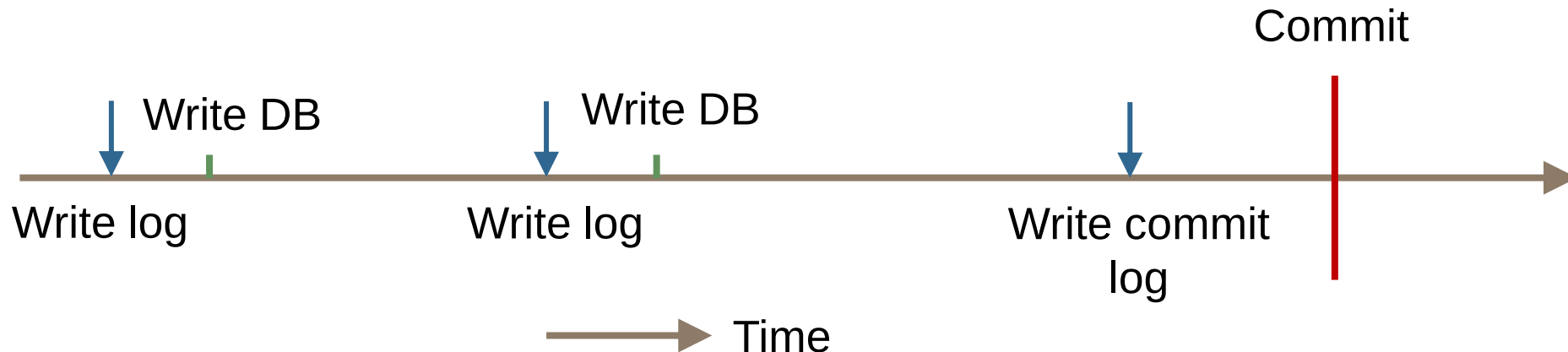
The WAL Rule for Log-Based Recovery



- WAL (Write Ahead Logging)
 - Before a modified page (or block) in main memory can be written to the database (non-volatile storage), "all its" log entries have to be written to stable storage



- Before a transaction enters the commit state, "all its" log entries have to be written to stable storage, incl. the commit log entry



Structure of A Log Entry (Log Record)



- [TID, DID, old, new]
 - **TID**: identifier of the transaction that caused the update
 - **DID**: data item identifier: location on **disk**
 - **old**: value of the data item before the update
 - **new**: value of the data item after the update
- Additional entries
 - **start**: Transaction TID has started [TID start]
 - **commit**: Transaction TID has committed [TID commit]
 - **abort**: Transaction TID has aborted [TID abort]

Log Entry Example



T_1	T_2
begin	
READ(B, b)	
$b \leftarrow b + 100$	
WRITE(B, b)	
commit	
	begin
	Read(D,d)
	Read(E,e)
	$d \leftarrow d - 10$
	WRITE(D,d)
	$e \leftarrow e - 20$
	WRITE(E,e)
	commit

Log

$[T_1, \text{start}]$

$[T_1, B, 300, 400]$

$[T_1, \text{commit}]$

$[T_2, \text{start}]$

$[T_2, D, 530, 520]$

$[T_2, E, 70, 50]$

$[T_2, \text{commit}]$

$[TID, DID, \text{old}, \text{new}]$



Should be written
before output(B)



Should be written
before commit T_1



Should be written
before output(D)



Should be written
before output(E)



Should be written
before commit T_2

Exercise 1



- The database initially contains the following values: $A=100$, $C=40$, $F=300$.
- Two transactions T_1 and T_2 are executed.
- Write all log records that would be generated if both transactions run correctly.

T_1	T_2
begin	
READ(A, a)	
$a \leftarrow a + 50$	
READ(C, c)	
$c \leftarrow c \times 2$	
WRITE(C, c)	
WRITE(A, a)	
commit	
	begin
	Read(F, f)
	$f \leftarrow f - 30$
	WRITE(F, f)
	commit

Exercise 1 -- Solution



- The database initially contains the following values: A=100, C=40, F=300.
- Two transactions T_1 and T_2 are executed.
- Write all log records that would be generated if both transactions run correctly.

T_1	T_2
begin	
READ(A, a)	
$a \leftarrow a + 50$	
READ(C, c)	
$c \leftarrow c \times 2$	
WRITE(C, c)	
WRITE(A, a)	
commit	
	begin
	Read(F, f)
	$f \leftarrow f - 30$
	WRITE(F, f)
	commit

Log

[T_1 , start]
[T_1 , C, 40, 80]
[T_1 , A, 100, 150]
[T_1 , commit]
[T_2 , start]
[T_2 , F, 300, 270]
[T_2 , commit]

Agenda

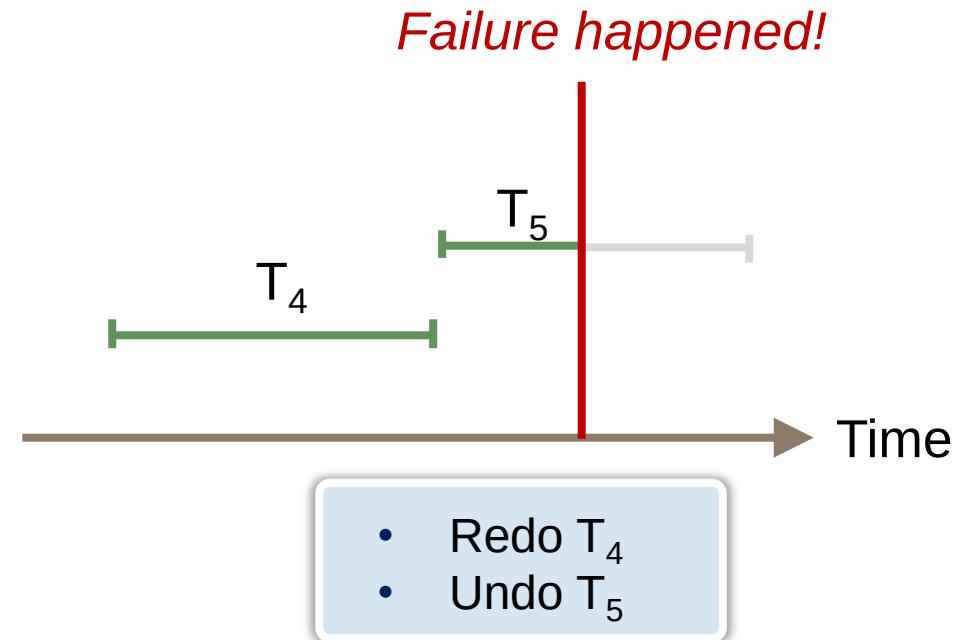
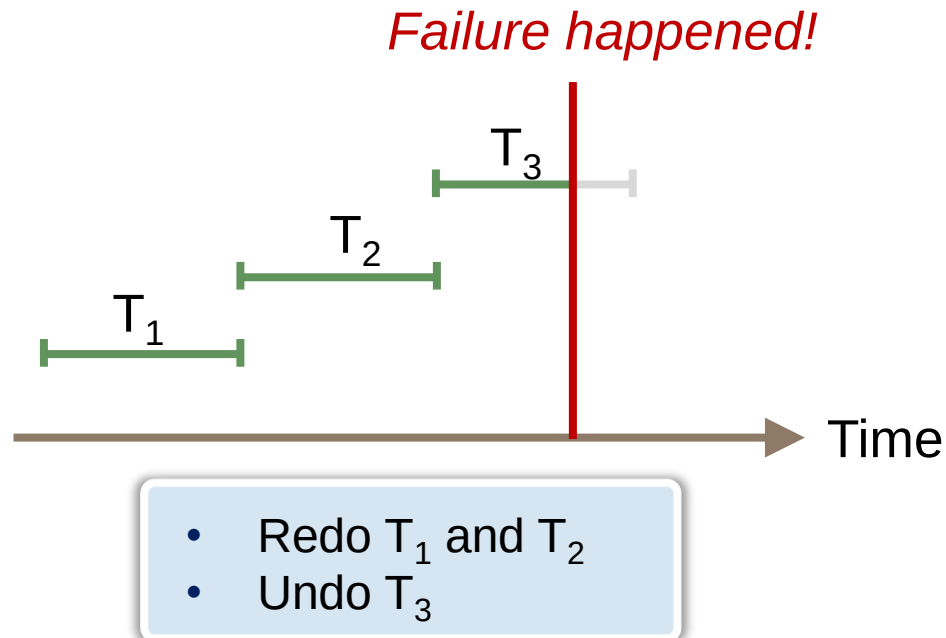


- Transaction Review
- Failure Classification
 - Transaction failure
 - System crash
 - Disk failure
- Log-Based Recovery
 - Log Entries
 - Recovery Process

Log-Based Recovery



- Operations to recover from failures
 - Redo: perform the changes to the database again
 - Undo: restore database to state prior to execution
- To recover from a failure
 - Redo results for committed transactions
 - Undo changes of transactions that did not commit



Log-Based Recovery

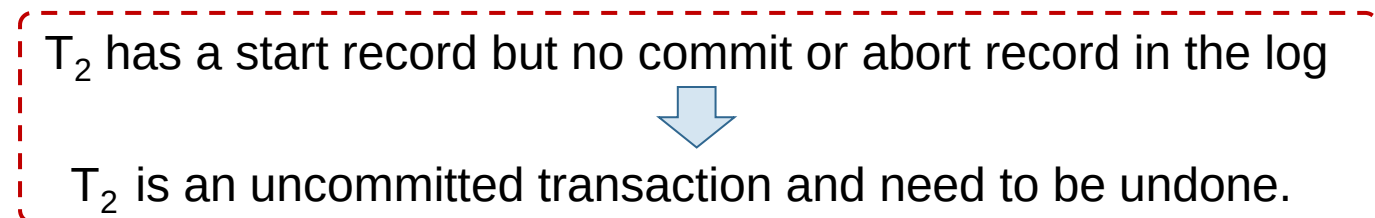
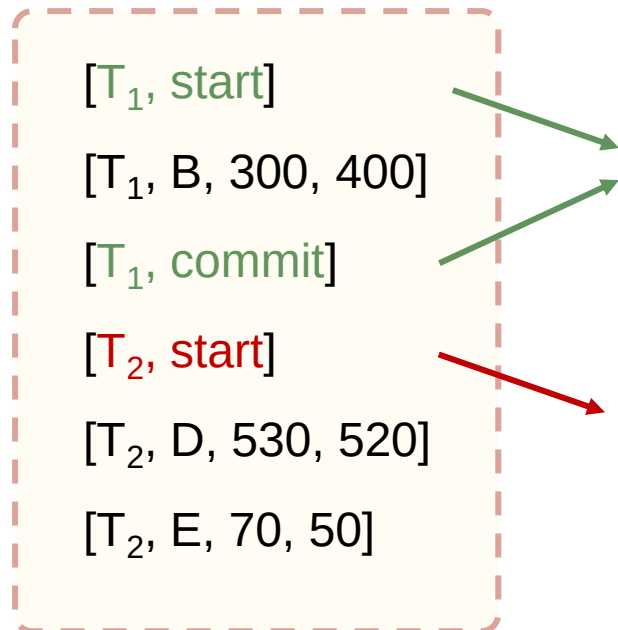


- Operations to recover from failures
 - Redo: perform the changes to the database again
 - Undo: restore database to state prior to execution
- To recover from a failure
 - Redo results for committed transactions
 - Undo changes of transactions that did not commit

Note

- *Redo does not mean executing the transaction again.*
- *It simply replays the committed updates recorded in the log to make sure the disk reflects the correct state.*

Log



The Phases of Recovery



- Redo (repeat history)
 - **Forward** scan through the log
 - Repeat **all** updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”
- Undo (rollback) all transactions in the “undo list”
 - **Backward** scan through the log
 - Undo all updates of transactions in the “undo list”: create a **compensating log record**
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Compensation Log Records



- [TID, DID, old]
 - Created to undo (compensate) the changes of [TID, DID, old, new]
 - Redo-only log record
 - Can also be used to rollback a transaction during **normal operation**

Example: Log-Based Recovery



- Assume a failure has occurred.
- The current state of the database and the log records are:

Database (Disk)	
A	100
B	200
C	50
D	60
E	120

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]

Log

- Use the log to perform log-based recovery.

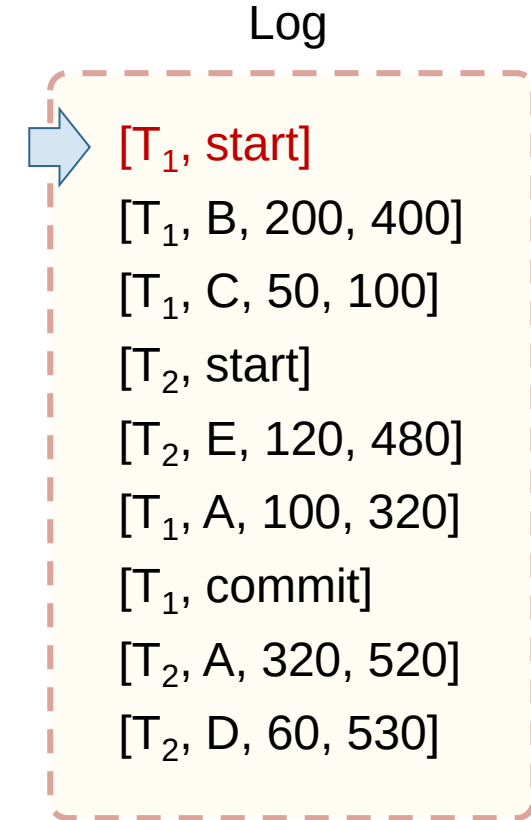
Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	200
C	50
D	60
E	120

- Undo list {}



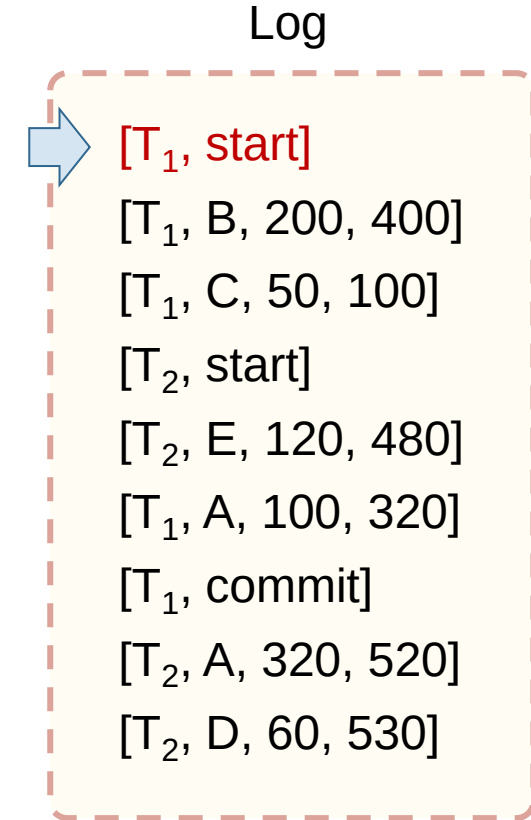
Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	200
C	50
D	60
E	120

- Undo list { T_1 }



Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	200 400
C	50
D	60
E	120

- Undo list {T₁}

Log

[T ₁ , start]
→ [T ₁ , B, 200, 400]
[T ₁ , C, 50, 100]
[T ₂ , start]
[T ₂ , E, 120, 480]
[T ₁ , A, 100, 320]
[T ₁ , commit]
[T ₂ , A, 320, 520]
[T ₂ , D, 60, 530]

Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	400
C	50 100
D	60
E	120

- Undo list {T₁}

Log

[T ₁ , start]
[T ₁ , B, 200, 400]
→ [T ₁ , C, 50, 100]
[T ₂ , start]
[T ₂ , E, 120, 480]
[T ₁ , A, 100, 320]
[T ₁ , commit]
[T ₂ , A, 320, 520]
[T ₂ , D, 60, 530]

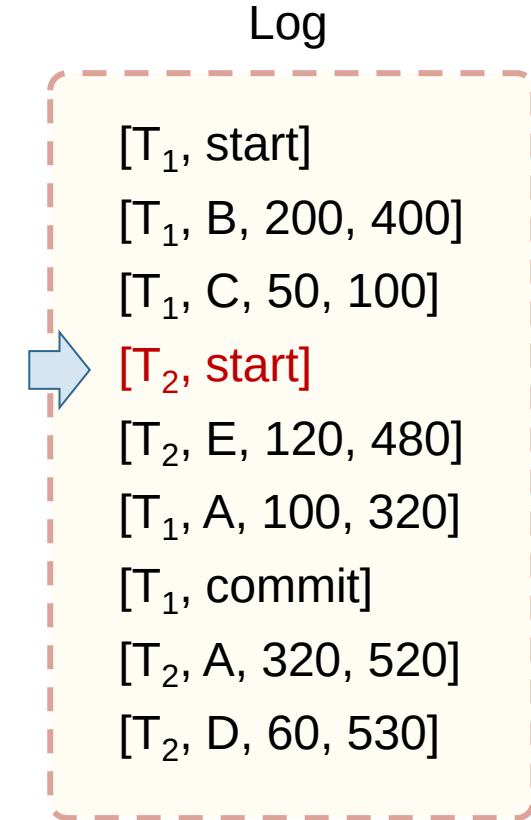
Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	400
C	100
D	60
E	120

- Undo list {T₁}



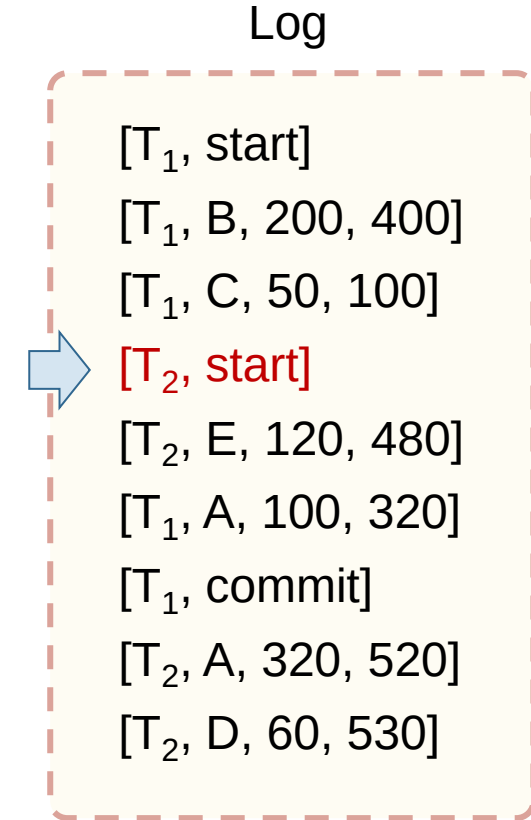
Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	400
C	100
D	60
E	120

- Undo list {T₁, T₂}



Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	400
C	100
D	60
E	120 480

- Undo list {T₁, T₂}

Log	
	[T ₁ , start]
	[T ₁ , B, 200, 400]
	[T ₁ , C, 50, 100]
	[T ₂ , start]
→	[T ₂ , E, 120, 480]
	[T ₁ , A, 100, 320]
	[T ₁ , commit]
	[T ₂ , A, 320, 520]
	[T ₂ , D, 60, 530]

Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100 320
B	400
C	100
D	60
E	480

- Undo list {T₁, T₂}

Log	
	[T ₁ , start]
	[T ₁ , B, 200, 400]
	[T ₁ , C, 50, 100]
	[T ₂ , start]
	[T ₂ , E, 120, 480]
→	[T ₁ , A, 100, 320]
	[T ₁ , commit]
	[T ₂ , A, 320, 520]
	[T ₂ , D, 60, 530]

Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	320
B	400
C	100
D	60
E	480

- Undo list {~~T₁~~, T₂}

Log

[T ₁ , start]
[T ₁ , B, 200, 400]
[T ₁ , C, 50, 100]
[T ₂ , start]
[T ₂ , E, 120, 480]
[T ₁ , A, 100, 320]
→ [T ₁ , commit]
[T ₂ , A, 320, 520]
[T ₂ , D, 60, 530]

Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	320 520
B	400
C	100
D	60
E	480

- Undo list {T₂}

Log	
[T ₁ , start]	
[T ₁ , B, 200, 400]	
[T ₁ , C, 50, 100]	
[T ₂ , start]	
[T ₂ , E, 120, 480]	
[T ₁ , A, 100, 320]	
[T ₁ , commit]	
[T ₂ , A, 320, 520]	
[T ₂ , D, 60, 530]	



Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	520
B	400
C	100
D	60 530
E	480

- Undo list {T₂}

Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
→ [T₂, D, 60, 530]

Example: Log-Based Recovery



- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	100
B	200
C	50
D	60
E	120

Before Phase 1

Database (Disk)	
A	520
B	400
C	100
D	530
E	480

After Phase 1

Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]

- Undo list {T₂}

Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	520
B	400
C	100
D	530
E	480

- Undo list { T_2 }

Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
[T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]

T_2 is in the undo list.



Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	520
B	400
C	100
D	530 60
E	480

- Undo list { T_2 }

T_2 is in the undo list.

Compensation log record



Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
[T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]
[T_2 , D, 60]

Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	520
B	400
C	100
D	60
E	480

- Undo list { T_2 }

T_2 is in the undo list.



Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
[T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]
[T_2 , D, 60]

Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	520 320
B	400
C	100
D	60
E	480

- Undo list { T_2 }

T_2 is in the undo list.

Compensation log record →

Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
[T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]
[T_2 , D, 60]
[T_2 , A, 320]

Example: Log-Based Recovery



- Phase 2 (undo)

- Backward scan through the log
- Undo all updates of transactions in the “undo list”: create a compensating log record
- For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
- Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	480

- Undo list {T₂}

T₁ isn't in the undo list.



Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]
[T₂, D, 60]
[T₂, A, 320]

Example: Log-Based Recovery



- Phase 2 (undo)

- Backward scan through the log
- Undo all updates of transactions in the “undo list”: create a compensating log record
- For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
- Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	480

- Undo list {T₂}

T₁ isn't in the undo list.



Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]
[T₂, D, 60]
[T₂, A, 320]

Example: Log-Based Recovery



- Phase 2 (undo)

- Backward scan through the log
- Undo all updates of transactions in the “undo list”: create a compensating log record
- For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
T₂ is in the undo list.
- Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	480

- Undo list {T₂}

Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]
[T₂, D, 60]
[T₂, A, 320]



Example: Log-Based Recovery



- Phase 2 (undo)

- Backward scan through the log
- Undo all updates of transactions in the “undo list”: create a compensating log record
- For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
T₂ is in the undo list.
- Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	480-120

- Undo list {T₂}

Log

[T₁, start]
[T₁, B, 200, 400]
[T₁, C, 50, 100]
[T₂, start]
[T₂, E, 120, 480]
[T₁, A, 100, 320]
[T₁, commit]
[T₂, A, 320, 520]
[T₂, D, 60, 530]
[T₂, D, 60]
[T₂, A, 320]
[T₂, E, 120]

Compensation log record →

Example: Log-Based Recovery

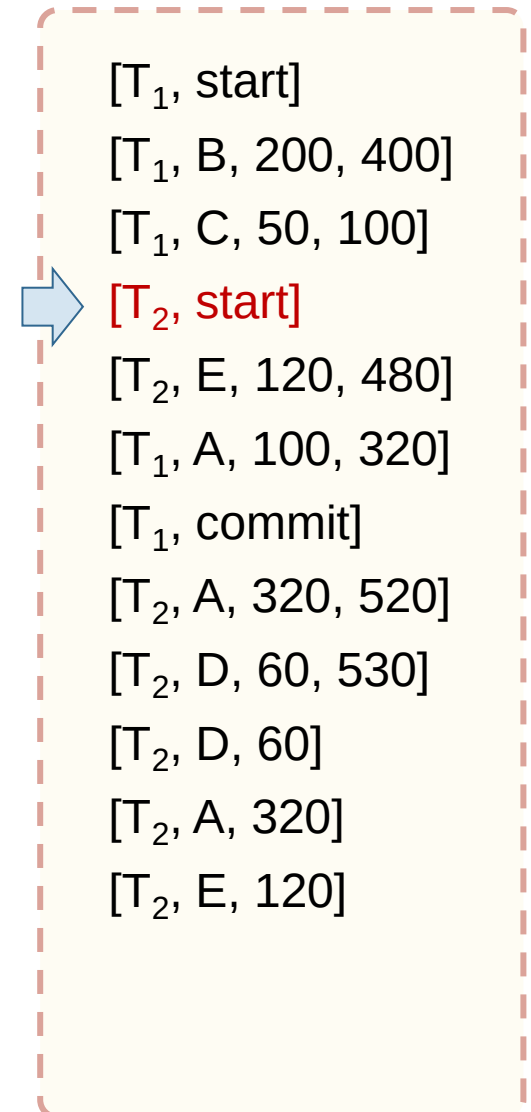


- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	120

- Undo list { T_2 }

Log



Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	120

- Undo list { T_2 }

Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
→ [T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]
[T_2 , D, 60]
[T_2 , A, 320]
[T_2 , E, 120]
Compensation log record → [T_2 , abort]

Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	120

- Undo list {}

Log

```
[T1, start]
[T1, B, 200, 400]
[T1, C, 50, 100]
[T2, start]
[T2, E, 120, 480]
[T1, A, 100, 320]
[T1, commit]
[T2, A, 320, 520]
[T2, D, 60, 530]
[T2, D, 60]
[T2, A, 320]
[T2, E, 120]
[T2, abort]
```


Example: Log-Based Recovery



- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	320
B	400
C	100
D	60
E	120

- Redo T_1
- Undo T_2

Log

[T_1 , start]
[T_1 , B, 200, 400]
[T_1 , C, 50, 100]
[T_2 , start]
[T_2 , E, 120, 480]
[T_1 , A, 100, 320]
[T_1 , commit]
[T_2 , A, 320, 520]
[T_2 , D, 60, 530]
[T_2 , D, 60]
[T_2 , A, 320]
[T_2 , E, 120]
[T_2 , abort]

Exercise 2



- Based on the given database state and log records, perform log-based recovery to restore consistency.

Database (Disk)	
A	500
B	20
C	70

Log

[T₁, start]
[T₁, B, 20, 25]
[T₂, start]
[T₂, C, 70, 60]
[T₂, commit]
[T₃, start]
[T₃, A, 500, 400]
[T₁, B, 20]
[T₁, abort]

Exercise 2 -- Solution



- Based on the given database state and log records, perform log-based recovery to restore consistency.
- Phase 1 (redo)
 - Forward scan through the log
 - Repeat all updates in the same order as in the log file
 - Determine “undo” transactions
 - ◆ [TID start] add TID to the “undo list”
 - ◆ [TID abort] or [TID commit] remove TID from the “undo list”

Database (Disk)	
A	500 400
B	20 25 20
C	70 60



Database (Disk)	
A	400
B	20
C	60

- Undo list { T_1 , T_2 , T_3 }

Log

[T_1 , start]
[T_1 , B, 20, 25]
[T_2 , start]
[T_2 , C, 70, 60]
[T_2 , commit]
[T_3 , start]
[T_3 , A, 500, 400]
[T_1 , B, 20]
[T_1 , abort]

Exercise 2 -- Solution



- Based on the given database state and log records, perform log-based recovery to restore consistency.
- Phase 2 (undo)
 - Backward scan through the log
 - Undo all updates of transactions in the “undo list”: create a compensating log record
 - For a [TID start] record of a transaction TID in the “undo list”, add a [TID abort] record to the log file, remove TID from the “undo list”
 - Stop, when “undo list” is empty

Database (Disk)	
A	400 500
B	20
C	60



Database (Disk)	
A	500
B	20
C	60

- Undo list { T_3 }

Log

[T_1 , start]
[T_1 , B, 20, 25]
[T_2 , start]
[T_2 , C, 70, 60]
[T_2 , commit]
[T_3 , start]
[T_3 , A, 500, 400]
[T_1 , B, 20]
[T_1 , abort]
[T_3 , A, 500]
[T_3 , abort]



Summary



- Transaction Review
- Failure Classification
 - Transaction failure
 - System crash
 - Disk failure
- Log-Based Recovery
 - Log Entries
 - Recovery Process



Any Questions?

Lecture 11
Recovery

Yumeng Song

yumengs@cs.aau.dk

Center for Data-Intensive Systems

Reference



- Abraham et al.. Database System Concepts (seventh edition) – Chapter 19
- Tiantian Liu. Database Systems(SW5) - 11. Recovery
- Christian S. Jensen. Database Systems - Transactions